

**SIMATS ENGINEERING
ASSESSMENT TOOL 3**

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

**COURSE CODE: CSA5117
NAME: G. BALASUBRAMANIAN
REG.NO:192521001
DEPARTMENT: B.TECH. IT
ASSIGNMENT: C03 – AT1
FACULTY NAME: DR. R. DHARANI**

1) Program: Comparison of MD5 and SHA-256 in Python

```
import hashlib
import time
# List of input messages
messages = [
    "Hello World",
    "Cryptography",
    "Network Security",
    "Python Programming",
    "Secure Hash Algorithm"
]
print("MD5 and SHA-256 Hash Comparison")
print("-" * 80)
# MD5 Hashing
start_md5 = time.time()
print("\nMD5 Hashes:")
for msg in messages:
    md5_hash = hashlib.md5(msg.encode()).hexdigest()
    print(f"Message : {msg}")
    print(f"MD5    : {md5_hash}\n")
end_md5 = time.time()
# SHA-256 Hashing
start_sha = time.time()
print("\nSHA-256 Hashes:")
for msg in messages:
    sha_hash = hashlib.sha256(msg.encode()).hexdigest()
    print(f"Message : {msg}")
    print(f"SHA-256 : {sha_hash}\n")
end_sha = time.time()
# Performance Comparison
md5_time = end_md5 - start_md5
sha_time = end_sha - start_sha
print("-" * 80)
print("Performance Comparison")
print(f"MD5 Time    : {md5_time:.8f} seconds")
print(f"SHA-256 Time : {sha_time:.8f} seconds")
if md5_time < sha_time:
    print("\nMD5 is faster than SHA-256.")
else:
    print("\nSHA-256 is faster than MD5.")
```

Sample Output

MD5 and SHA-256 Hash Comparison

Message : Hello World

MD5 : b10a8db164e0754105b7a99be72e3fe5

SHA-256 : a591a6d40bf420404a011733cfb7b190...

(The remaining messages will produce their respective MD5 and SHA-256 hashes.)

Comparison of MD5 and SHA-256

Feature	MD5	SHA-256
Hash Length	128 bits (32 hex characters)	256 bits (64 hex characters)
Speed	Faster	Slightly slower
Collision Resistance	Weak	Strong
Security	Broken and obsolete	Secure and widely used
Password Storage	Not recommended	Better (with salting); bcrypt/Argon2 preferred
Common Uses	File checksum (legacy)	SSL/TLS, Digital Signatures, Blockchain

Collision Resistance Analysis

MD5

- Produces a **128-bit** hash.
- Vulnerable to **collision attacks**, where two different inputs can produce the same hash.
- Not suitable for password hashing, digital signatures, or secure authentication.

SHA-256

- Produces a **256-bit** hash.
- Provides much stronger collision resistance.
- No practical collision attacks are currently known.
- Recommended for modern cryptographic applications.

Performance Analysis

- **MD5** executes faster because it performs fewer cryptographic operations and generates a shorter hash.
- **SHA-256** takes slightly more time due to its stronger security and longer hash length.
- The performance difference is generally small on modern computers, while the security improvement is substantial.

Conclusion

- **MD5** is faster but insecure because it is vulnerable to collision attacks and should not be used for security-critical applications.
- **SHA-256** is slightly slower but offers significantly stronger security and collision resistance, making it the preferred choice for data integrity, digital signatures, and secure communication.
- For password storage, modern algorithms such as **bcrypt**, **scrypt**, or **Argon2** are recommended over both MD5 and plain SHA-256.

2) Python Program: RSA Digital Signature (Message Signing & Verification)

```
from Crypto.PublicKey import RSA
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256
key = RSA.generate(2048)
private_key = key
public_key = key.publickey()
print("RSA Key Pair Generated Successfully\n")
message = b"Welcome to Cryptography and Network Security"
print("Original Message:")
print(message.decode())
hash_obj = SHA256.new(message)
signature = pkcs1_15.new(private_key).sign(hash_obj)
print("\nDigital Signature (Hex):")
print(signature.hex())
try:
    pkcs1_15.new(public_key).verify(hash_obj, signature)
    print("\nSignature Verification: SUCCESS")
    print("Message is Authentic and Integrity is Maintained.")
except (ValueError, TypeError):
    print("\nSignature Verification: FAILED")
    print("Message has been modified.")
tampered_message = b"Welcome to Cyber Security"
tampered_hash = SHA256.new(tampered_message)
print("\nTesting Tampered Message...")
try:
    pkcs1_15.new(public_key).verify(tampered_hash, signature)
    print("Signature Valid")
except (ValueError, TypeError):
    print("Signature Invalid")
    print("Tampering Detected!")
```

Sample Output

```
RSA Key Pair Generated Successfully
Original Message:
Welcome to Cryptography and Network Security
Digital Signature (Hex):
5d8f6a9b8f3d6e4c9a1b7...
Signature Verification: SUCCESS
Message is Authentic and Integrity is Maintained.
Testing Tampered Message...
Signature Invalid
Tampering Detected!
```

Working Principle

1. Generate an RSA public and private key pair.
2. Create a SHA-256 hash of the original message.
3. Sign the hash using the sender's **private key**.
4. Send the message along with its digital signature.
5. The receiver hashes the received message using SHA-256.
6. The receiver verifies the signature using the sender's **public key**.
7. If verification succeeds, the message is authentic and unaltered; otherwise, it has been modified or is not from the claimed sender.

Analysis of Effectiveness

Integrity

- Any change to the message produces a different SHA-256 hash.
- Signature verification fails if the message is modified, ensuring data integrity.

Authentication

- Only the sender possessing the private key can generate a valid signature.
- The receiver verifies it using the sender's public key, confirming the sender's identity.

Non-Repudiation

- Since only the sender owns the private key, the sender cannot deny having signed the message.

Security

- RSA with a **2048-bit key** and **SHA-256** provides strong cryptographic security.
- Widely used in SSL/TLS, secure email, software signing, and digital certificates.

Advantages

- Ensures **message integrity**.
- Provides **sender authentication**.
- Supports **non-repudiation**.
- Detects message tampering.
- Secure with modern RSA key sizes and SHA-256.

Limitations

- RSA signature generation is slower than symmetric cryptography.
- Larger key sizes require more computation.
- The private key must be securely protected.
- A trusted Public Key Infrastructure (PKI) is needed to validate public keys.

Conclusion

The RSA Digital Signature scheme successfully signs and verifies messages using **SHA-256** and **RSA-2048**. It effectively guarantees **integrity**, **authentication**, and **non-repudiation**. Any modification to the message results in signature verification failure, making RSA digital signatures a reliable choice for secure communication and electronic transactions.

3) Python Program: HMAC Using SHA-256

```
import hashlib
import hmac
secret_key = b"SecureKey123"
message = b"Welcome to Cryptography"
simple_hash = hashlib.sha256(message).hexdigest()
hmac_hash = hmac.new(secret_key, message, hashlib.sha256).hexdigest()
print("Original Message:")
print(message.decode())
print("\nSimple SHA-256 Hash:")
print(simple_hash)
print("\nHMAC (SHA-256):")
print(hmac_hash)
received_message = message
received_hmac = hmac.new(secret_key, received_message,
hashlib.sha256).hexdigest()
print("\nVerification:")
if hmac.compare_digest(hmac_hash, received_hmac):
    print("Message is Authentic and Integrity Maintained.")
else:
    print("Message Modified!")
tampered_message = b"Welcome to Cyber Security"
tampered_hmac = hmac.new(secret_key, tampered_message,
hashlib.sha256).hexdigest()
print("\nTesting Tampered Message:")
if hmac.compare_digest(hmac_hash, tampered_hmac):
    print("Message Authentic")
else:
    print("Tampering Detected!")
```

Sample Output

```
Original Message:
Welcome to Cryptography
Simple SHA-256 Hash:
6fd0fdf8f8dcb9d2b9d6d2a9e1d8...
HMAC (SHA-256):
2bcb1db8c86f5d2d7a9e4b83f1d4...
Verification:
Message is Authentic and Integrity Maintained.
Testing Tampered Message:
Tampering Detected!
```

Working Principle

1. A **shared secret key** is generated.
2. The sender combines the **secret key** with the message.
3. HMAC uses **SHA-256** to generate a Message Authentication Code (MAC).
4. The sender transmits the message along with the HMAC.

Security Comparison: HMAC vs Simple Hash

Feature	Simple SHA-256 Hash	HMAC (SHA-256)
Secret Key Required	No	Yes
Message Integrity	Yes	Yes
Sender Authentication	No	Yes
Prevents Message Tampering	No	Yes
Resistant to Forgery	No	Yes
Suitable for Secure Communication	No	Yes
Common Applications	File integrity checking	APIs, Banking, SSL/TLS, VPNs

Analysis

Simple Hash

- Verifies only **message integrity**.
- Anyone can modify the message and recompute the hash.
- Does **not** authenticate the sender.
- Vulnerable to message forgery.

HMAC

- Uses a **shared secret key** with the hash function.
 - Only authorized users who know the secret key can generate a valid HMAC.
 - Provides both **integrity** and **authentication**.
 - Prevents message tampering and forgery.
-

Advantages of HMAC

- Ensures message integrity.
 - Authenticates the sender.
 - Protects against unauthorized message modification.
 - Resistant to forgery without the secret key.
 - Efficient and widely used in secure communication protocols.
-

Conclusion

A simple hash function (such as SHA-256) can detect accidental changes but cannot verify the sender because anyone can generate a new hash. **HMAC-SHA-256** overcomes this limitation by incorporating a **shared secret key**, ensuring both **message integrity** and **authentication**. For this reason, HMAC is widely used in secure applications such as **TLS/SSL, REST APIs, VPNs, cloud services, and online banking**.

4) Python Program: Simplified Kerberos Authentication Mechanism

```
import time
import secrets

class KDC:
    def __init__(self):
        self.secret = "KDC_SECRET"
    def generate_ticket(self, client_name):
        ticket = {
            "client": client_name,
            "session_key": secrets.token_hex(8),
            "timestamp": time.time(),
            "lifetime": 30    # Ticket valid for 30 seconds
        }
        return ticket

class Client:
    def __init__(self, name):
        self.name = name
    def request_ticket(self, kdc):
        print("Client requesting ticket from KDC...")
        ticket = kdc.generate_ticket(self.name)
        print("Ticket Received\n")
        return ticket

class Server:
    def __init__(self):
        self.used_timestamps = []
    def validate_ticket(self, ticket):
        current_time = time.time()
        # Check ticket expiry
        if current_time - ticket["timestamp"] > ticket["lifetime"]:
            print("Ticket Expired!")
            return False
        # Replay attack detection
        if ticket["timestamp"] in self.used_timestamps:
            print("Replay Attack Detected!")
            return False
        self.used_timestamps.append(ticket["timestamp"])
        print("Authentication Successful")
        print("Client :", ticket["client"])
        print("Session Key :", ticket["session_key"])
        return True

kdc = KDC()
client = Client("Alice")
server = Server()
# Ticket Generation
ticket = client.request_ticket(kdc)
print("First Authentication")
server.validate_ticket(ticket)
```

```
print("\nTrying Replay Attack...")
server.validate_ticket(ticket)
```

Sample Output

Client requesting ticket from KDC...
Ticket Received
First Authentication
Authentication Successful
Client : Alice
Session Key : 7fa3b6d92a4c5e81
Trying Replay Attack...
Replay Attack Detected!

Working Principle

Step 1 – Client Requests Authentication

The client sends a request to the **Key Distribution Center (KDC)**.

Step 2 – Ticket Generation

The KDC generates a ticket containing:

- Client ID
- Session Key
- Timestamp
- Ticket Lifetime

Step 3 – Ticket Distribution

The KDC sends the ticket securely to the client.

Step 4 – Client Accesses Server

The client presents the ticket to the server.

Step 5 – Ticket Validation

The server verifies:

- Ticket validity
- Ticket expiration
- Timestamp
- Whether the ticket has already been used

Step 6 – Replay Attack Detection

If the same ticket is presented again, the server rejects it as a replay attack.

Effectiveness in Preventing Replay Attacks

Strengths

- **Timestamp Validation:** Ensures tickets are accepted only within a limited validity period.
- **Short Ticket Lifetime:** Reduces the time available for attackers to reuse intercepted tickets.
- **Replay Detection:** The server stores previously used timestamps and rejects duplicate tickets.
- **Session Keys:** Temporary session keys are generated for each authentication session, limiting exposure.

Limitations

- This is a simplified simulation and does not include:
 - Encryption of tickets
 - Ticket Granting Tickets (TGT)
 - Mutual authentication
 - Network communication
 - Time synchronization between systems
- In real Kerberos, accurate clock synchronization is essential; significant clock differences can cause valid tickets to be rejected or expired tickets to be accepted.

Advantages

- Prevents replay attacks using timestamps and single-use ticket validation.
- Supports secure authentication without transmitting passwords repeatedly.
- Uses temporary session keys for secure communication.
- Demonstrates the core concepts of the Kerberos protocol.

Conclusion

This Python program demonstrates a **simplified Kerberos authentication mechanism** by simulating **ticket generation, distribution, and validation** between a client and server. The use of **timestamps, ticket expiration, and replay detection** effectively prevents replay attacks in the simulation. In real-world Kerberos deployments, additional features such as encrypted tickets, Ticket Granting Tickets (TGTs), mutual authentication, and synchronized system clocks provide stronger security for enterprise networks.

5) Below is a complete **Python implementation of the ElGamal Digital Signature Scheme**, including key generation, signing, verification, sample output, and a security comparison.

1. Objective

To implement the **ElGamal digital signature scheme** in Python and evaluate its security features compared with standard digital signature algorithms such as **RSA** and **ECDSA**.

2. ElGamal Digital Signature

ElGamal signatures are based on the difficulty of solving the **Discrete Logarithm Problem (DLP)**.

The algorithm consists of three stages:

1. **Key Generation**
2. **Signature Generation**
3. **Signature Verification**

For a prime (p), choose a generator (g) and private key (x).

The public key is:

```
[  
y=g^x \mod p  
]
```

For message (M), calculate:

```
[  
h=H(M)  
]
```

Choose a random (k) such that:

```
[  
1<k<p-1, \text{gcd}(k,p-1)=1  
]
```

Calculate:

```
[  
r=g^k \mod p  
]
```

and

```
[  
s=k^{-1}(h-xr) \mod (p-1)  
]
```

The signature is:

```
[  
(r,s)  
]
```

Verification checks:

```
[  
g^h \equiv y^r r^s \pmod p  
]
```

3. Python Implementation

```
import hashlib
import math
import secrets

def mod_inverse(a, m):
    old_r, r = a, m
    old_s, s = 1, 0
    while r != 0:
        q = old_r // r
        old_r, r = r, old_r - q * r
        old_s, s = s, old_s - q * s
    if old_r != 1:
        raise ValueError("Modular inverse does not exist")
    return old_s % m

def hash_message(message):
    digest = hashlib.sha256(message.encode()).hexdigest()
    return int(digest, 16)

def generate_keys():
    p = 467
    g = 2
    x = secrets.randbelow(p - 2) + 1
    y = pow(g, x, p)
    private_key = (p, g, x)
    public_key = (p, g, y)
    return private_key, public_key

def sign(message, private_key):
    p, g, x = private_key
    h = hash_message(message) % (p - 1)
    while True:
        k = secrets.randbelow(p - 2) + 2
        if math.gcd(k, p - 1) == 1:
            break
    r = pow(g, k, p)
    k_inverse = mod_inverse(k, p - 1)
    s = (k_inverse * (h - x * r)) % (p - 1)
    return r, s

def verify(message, signature, public_key):
    p, g, y = public_key
    r, s = signature
    if not (0 < r < p):
        return False
    if not (0 < s < p - 1):
        return False
    h = hash_message(message) % (p - 1)
    left = pow(g, h, p)
    right = (pow(y, r, p) * pow(r, s, p)) % p
```

```

    return left == right
print("ELGAMAL DIGITAL SIGNATURE")
print("=" * 40)
private_key, public_key = generate_keys()
print("\nPrivate Key:")
print(private_key)
print("\nPublic Key:")
print(public_key)
message = "This is a confidential message."
print("\nOriginal Message:")
print(message)
signature = sign(message, private_key)
print("\nDigital Signature:")
print(signature)
result = verify(message, signature, public_key)
print("\nSignature Verification:")
if result:
    print("VALID SIGNATURE")
else:
    print("INVALID SIGNATURE")
modified_message = "This is a modified message."
result_modified = verify(
    modified_message,
    signature,
    public_key
)
print("\nModified Message Verification:")
if result_modified:
    print("VALID SIGNATURE")
else:
    print("INVALID SIGNATURE")

```

4. Sample Output

The private key and signature contain random values, so the exact output changes every execution.

ELGAMAL DIGITAL SIGNATURE

Private Key:

(467, 2, 183)

Public Key:

(467, 2, 325)

Original Message:

This is a confidential message.

Digital Signature:

(231, 117)

Signature Verification:

VALID SIGNATURE

Modified Message Verification:

INVALID SIGNATURE

5. Working of the Program

Key Generation:

- A prime number (p) and generator (g) are selected.
- A random private key (x) is generated.
- The public key is calculated as:

[
 $y = g^x \bmod p$
]

Signing:

The SHA-256 hash of the message is calculated. A fresh random value (k) is selected for every signature. The program then calculates (r) and (s), producing the digital signature ((r,s)).

Verification:

The receiver calculates both sides of:

[
 $g^h \equiv y^r r^s \bmod p$
]

If they are equal, the signature is valid.

When even one character of the message is changed, its hash changes, causing verification to fail.

6. Security Evaluation

Feature	ElGamal	RSA	ECDSA
Mathematical basis	Discrete logarithm	Integer factorization	Elliptic-curve discrete logarithm
Authentication	Yes	Yes	Yes
Integrity	Yes	Yes	Yes
Non-repudiation	Yes	Yes	Yes
Random value required for signing	Yes	Generally no	Yes
Key size	Large	Large	Much smaller
Computational efficiency	Moderate	Moderate	High
Signature size	Relatively large	Moderate	Small
Main security risk	Reusing/weak (k)	Weak RSA parameters/private key	Reusing/weak nonce

7. Security Features

Integrity:

A modification to the original message produces a different hash, causing signature verification to fail.

Authentication:

Only someone possessing the private key can generate a valid signature.

Non-repudiation:

The signature provides evidence that the message was signed using the corresponding private key, assuming the private key remains under the owner's control.

Protection against forgery:

An attacker cannot practically create a valid signature without solving the underlying discrete logarithm problem or obtaining the private key when secure parameters are used.

Randomness requirement:

ElGamal requires a fresh, unpredictable (k) for every signature. Reusing or predicting (k) can expose the private key. This is one of the most important implementation considerations.

8. Comparison with Standard Digital Signatures

ElGamal provides strong security when implemented with sufficiently large parameters and secure randomness. However, the basic ElGamal signature scheme is generally **less attractive for modern applications** because its keys and signatures are relatively large.

RSA is widely deployed and has well-established standards, but requires substantially larger keys than elliptic-curve schemes for comparable security.

ECDSA provides comparable security with much smaller keys and signatures, making it more suitable for mobile devices, embedded systems, and bandwidth-constrained environments.

For a practical modern system, standardized implementations such as **RSA-PSS** or **ECDSA** are preferable to implementing textbook ElGamal directly.

9. Conclusion

The Python implementation successfully demonstrates the **ElGamal digital signature scheme**. The experiment shows that the scheme can provide **message integrity, authentication, and non-repudiation**. Any modification of the signed message results in verification failure.

However, the security of ElGamal depends heavily on secure parameter selection and, especially, the generation of a **unique and unpredictable random (k)** for every signature. Compared with RSA and ECDSA, ElGamal has larger signatures and less efficient parameter requirements. Therefore, while it is valuable for understanding public-key digital signatures, **RSA-PSS and ECDSA are generally better choices for modern production systems**.